



UNIVERSIDAD DON BOSCO

Facultad de Ingeniería

Ingeniería en Ciencias de la Computación

Proyecto 1

Sistema de Carrito de Compras - Walmart Latam

Presentado por:

Téc. Isidro Alexander Marroquín Echeverría

ME221443

Asignatura:

Desarrollo de Software para Móviles

DSM941 G01T

Docente:

Ing. Alexander Alberto Siguenza Campos

Fecha de entrega: 29 de marzo de 2026

Repositorio GitHub:

`https://github.com/marroquin9953/Proyecto-1---DSM941`

Índice

1. Resumen Ejecutivo	7
2. Introducción	7
2.1. Contexto del Proyecto	7
2.2. Objetivos	7
2.2.1. Objetivo General	7
2.2.2. Objetivos Específicos	8
2.3. Alcance	8
3. Tecnologías Utilizadas	9
3.1. Lenguaje y Plataforma	9
3.1.1. Kotlin 2.0.0	9
3.1.2. JVM 21	9
3.2. Sistema de Construcción	10
3.2.1. Gradle 8.10.1	10
3.3. Bibliotecas y Dependencias	10
3.3.1. AsciiTable 0.3.2	10
3.3.2. Gson 2.10.1	10
3.3.3. Kotlinx DateTime 0.6.0	11
3.3.4. iText7 Core 7.2.5	11
3.3.5. JavaMail 1.6.2	11
3.3.6. JUnit 5 y Kotlin Test	12
3.3.7. Shadow Plugin 7.1.2	12
4. Arquitectura del Sistema	12
4.1. Estructura General	12
4.2. Componentes Principales	13

4.2.1.	App.kt - Punto de Entrada	13
4.2.2.	Modelos de Datos	13
4.2.3.	Carrito.kt - Gestión del Carrito	14
4.2.4.	GestorInventario.kt - Persistencia de Inventario	14
4.2.5.	GestorHistorial.kt - Registro de Compras	14
4.2.6.	GeneradorFactura.kt - Facturas TXT/HTML	15
4.2.7.	GeneradorPDF.kt - Facturas PDF	15
4.2.8.	ServicioCorreo.kt - Envío de Correos	16
4.2.9.	Logger.kt - Sistema de Logging	16
4.2.10.	Validador.kt - Validaciones	16
4.2.11.	Excepciones.kt - Manejo de Errores	17

5. Funcionalidades Implementadas 17

5.1.	Registro de Clientes	17
5.2.	Catálogo de Productos	18
5.3.	Gestión del Carrito	18
5.3.1.	Agregar Productos	18
5.3.2.	Eliminar Productos	19
5.4.	Proceso de Checkout	19
5.5.	Generación de Facturas	20
5.5.1.	Factura TXT	20
5.5.2.	Factura HTML	20
5.5.3.	Factura PDF	21
5.6.	Envío de Correos	21
5.7.	Persistencia de Datos	22
5.7.1.	Inventario (data/inventario.json)	22
5.7.2.	Historial (data/historial_compras.json)	22
5.8.	Sistema de Logging	23

6. Principios de Diseño Aplicados	24
6.1. Principios SOLID	24
6.1.1. Single Responsibility Principle (SRP)	24
6.1.2. Open/Closed Principle (OCP)	24
6.1.3. Liskov Substitution Principle (LSP)	24
6.1.4. Interface Segregation Principle (ISP)	25
6.1.5. Dependency Inversion Principle (DIP)	25
6.2. Patrones de Diseño	25
6.2.1. Repository Pattern	25
6.2.2. Factory Pattern	25
6.2.3. Builder Pattern	26
6.2.4. Singleton Pattern	26
7. Manejo de Excepciones	26
7.1. Excepciones Personalizadas	26
7.2. Estrategia de Manejo	27
7.3. Validaciones Preventivas	27
8. Requisitos y Configuración	28
8.1. Requisitos del Sistema	28
8.1.1. Hardware	28
8.1.2. Software	28
8.2. Instalación	28
8.2.1. Clonar el Repositorio	28
8.2.2. Verificar Java	28
8.2.3. Dar Permisos (Linux/macOS)	28
8.3. Compilación	29
8.3.1. Compilar el Proyecto	29

8.3.2. Generar JAR Ejecutable	29
8.4. Ejecución	29
8.4.1. Opción 1: Con Gradle	29
8.4.2. Opción 2: JAR Directo	29
8.4.3. Opción 3: Desde IDE	29
8.5. Configuración de Correo (Opcional)	30
9. Pruebas	30
9.1. Pruebas Unitarias	30
9.1.1. Ejecutar Pruebas	30
9.1.2. Ver Reporte	30
9.2. Áreas de Prueba	31
10. Flujo de Uso del Sistema	32
10.1. Diagrama de Flujo	32
10.2. Ejemplo de Uso	33
10.2.1. Escenario: Compra de Laptop y Mouse	33
11. Limitaciones y Mejoras Futuras	33
11.1. Limitaciones Conocidas	33
11.2. Mejoras Futuras	34
11.2.1. Corto Plazo	34
11.2.2. Mediano Plazo	34
11.2.3. Largo Plazo	35

1. Resumen Ejecutivo

El presente documento describe el desarrollo de un Sistema de Carrito de Compras para Walmart Latam, implementado en Kotlin como parte del Proyecto 1 de Cátedra de la asignatura Desarrollo de Software para Móviles (DSM941) en la Universidad Don Bosco.

El sistema permite a los clientes navegar por un catálogo de productos, agregar artículos al carrito, gestionar sus compras y generar facturas en múltiples formatos (TXT, HTML, PDF). El proyecto implementa principios de programación orientada a objetos, manejo de excepciones, persistencia de datos en JSON, generación de documentos y logging de operaciones.

El desarrollo se realizó utilizando Kotlin 2.0.0 sobre JVM 21, con Gradle 8.10.1 como sistema de construcción. Se integraron bibliotecas especializadas como iText7 para generación de PDF, Gson para persistencia JSON, y JavaMail para envío de correos electrónicos.

2. Introducción

2.1. Contexto del Proyecto

En el contexto actual del comercio electrónico, los sistemas de carrito de compras son componentes fundamentales para la experiencia del usuario en plataformas de venta en línea. Este proyecto simula un sistema de gestión de compras para Walmart Latam, implementando las funcionalidades esenciales que permiten a los clientes realizar transacciones de manera eficiente y segura.

2.2. Objetivos

2.2.1. Objetivo General

Desarrollar un sistema de carrito de compras funcional en Kotlin que implemente las mejores prácticas de programación orientada a objetos y permita gestionar el ciclo completo de una

compra en línea.

2.2.2. Objetivos Específicos

- Implementar un sistema de gestión de inventario con persistencia en JSON
- Desarrollar un carrito de compras con validación de stock en tiempo real
- Generar facturas en múltiples formatos (TXT, HTML, PDF)
- Implementar un sistema de logging para auditoría de operaciones
- Aplicar principios SOLID y patrones de diseño
- Validar entradas de usuario y manejar excepciones de manera robusta

2.3. Alcance

El sistema cubre las siguientes funcionalidades:

- Registro de clientes con validación de datos
- Visualización de catálogo de productos con stock disponible
- Gestión de carrito de compras (agregar/eliminar productos)
- Proceso de checkout con confirmación
- Generación automática de facturas en tres formatos
- Envío opcional de facturas por correo electrónico
- Persistencia de inventario y historial de compras
- Sistema de logging para trazabilidad

3. Tecnologías Utilizadas

3.1. Lenguaje y Plataforma

3.1.1. Kotlin 2.0.0

Kotlin es un lenguaje de programación moderno, conciso y seguro que se ejecuta sobre la JVM. Fue seleccionado por las siguientes razones:

- **Null Safety:** Sistema de tipos que previene errores de NullPointerException
- **Concisión:** Reduce significativamente el código boilerplate comparado con Java
- **Interoperabilidad:** Compatible con todas las bibliotecas Java existentes
- **Funciones de extensión:** Permite extender clases sin herencia
- **Data classes:** Simplifica la creación de modelos de datos
- **Coroutines:** Soporte nativo para programación asíncrona

3.1.2. JVM 21

Java Virtual Machine versión 21 proporciona:

- Mejoras de rendimiento y optimización
- Recolección de basura mejorada
- Soporte para características modernas del lenguaje
- Estabilidad y madurez probada en producción

3.2. Sistema de Construcción

3.2.1. Gradle 8.10.1

Gradle es el sistema de construcción utilizado, ofreciendo:

- Gestión declarativa de dependencias
- Builds incrementales para mayor velocidad
- Soporte nativo para Kotlin DSL
- Integración con repositorios Maven Central
- Plugins para empaquetado y distribución

3.3. Bibliotecas y Dependencias

3.3.1. AsciiTable 0.3.2

Biblioteca para renderizado de tablas formateadas en consola, utilizada para:

- Visualización del catálogo de productos
- Presentación del contenido del carrito
- Formato de facturas en texto plano

3.3.2. Gson 2.10.1

Biblioteca de Google para serialización y deserialización JSON:

- Persistencia del inventario de productos
- Almacenamiento del historial de compras

- Conversión automática entre objetos Kotlin y JSON
- Soporte para tipos genéricos y colecciones

3.3.3. Kotlinx DateTime 0.6.0

Biblioteca oficial de Kotlin para manejo de fechas:

- Generación de timestamps para facturas
- Registro de fechas en historial de compras
- Formato de fechas en logs
- Multiplataforma y type-safe

3.3.4. iText7 Core 7.2.5

Biblioteca profesional para generación de documentos PDF:

- Creación de facturas en formato PDF
- Soporte para tablas, estilos y formato
- Generación de documentos de calidad profesional
- Ampliamente utilizada en entornos empresariales

3.3.5. JavaMail 1.6.2

API estándar de Java para envío de correos electrónicos:

- Envío de facturas por correo electrónico
- Soporte para SMTP con autenticación

- Adjuntos de archivos (facturas PDF)
- Configuración mediante archivos properties

3.3.6. JUnit 5 y Kotlin Test

Frameworks de testing para garantizar calidad del código:

- Pruebas unitarias de componentes
- Assertions específicas de Kotlin
- Integración con Gradle para ejecución automatizada
- Reportes de cobertura de código

3.3.7. Shadow Plugin 7.1.2

Plugin de Gradle para empaquetado:

- Generación de JAR ejecutable con todas las dependencias
- Simplifica la distribución del sistema
- Resolución automática de conflictos de dependencias

4. Arquitectura del Sistema

4.1. Estructura General

El sistema sigue una arquitectura modular basada en capas, separando responsabilidades en componentes especializados:

- **Capa de Presentación:** Interfaz de consola con menús y tablas

- **Capa de Lógica de Negocio:** Gestión de carrito, validaciones y procesamiento
- **Capa de Persistencia:** Gestores de inventario e historial
- **Capa de Servicios:** Generación de documentos y envío de correos
- **Capa de Utilidades:** Logging, validaciones y funciones auxiliares

4.2. Componentes Principales

4.2.1. App.kt - Punto de Entrada

Clase principal que orquesta el flujo de la aplicación:

- Inicialización del sistema y carga de inventario
- Captura y validación de datos del cliente
- Navegación del catálogo de productos
- Gestión del ciclo de vida del carrito
- Proceso de checkout y confirmación
- Generación y envío de facturas

4.2.2. Modelos de Datos

Cliente.kt: Representa un cliente con nombre y correo electrónico. Incluye validaciones de formato y métodos para obtener representaciones formales.

Producto.kt: Clase base que define las propiedades fundamentales de un producto (id, nombre, precio).

ProductoInventario.kt: Extiende Producto agregando información de stock disponible.

ProductoCarrito.kt: Representa un producto en el carrito con cantidad seleccionada y métodos para calcular subtotales.

4.2.3. Carrito.kt - Gestión del Carrito

Componente central que maneja la lógica del carrito de compras:

- Agregar productos con validación de stock
- Eliminar productos parcial o totalmente
- Calcular totales y subtotales
- Mostrar contenido en formato tabla
- Validar disponibilidad antes de checkout

4.2.4. GestorInventario.kt - Persistencia de Inventario

Implementa el patrón Repository para gestionar el inventario:

- Carga inicial del inventario desde JSON
- Actualización de stock después de compras
- Persistencia de cambios en archivo JSON
- Creación de inventario inicial si no existe
- Validación de integridad de datos

4.2.5. GestorHistorial.kt - Registro de Compras

Mantiene un historial completo de todas las transacciones:

- Registro de cada compra con timestamp

- Almacenamiento de datos del cliente
- Detalle de productos comprados
- Totales y subtotales
- Persistencia en formato JSON

4.2.6. GeneradorFactura.kt - Facturas TXT/HTML

Genera facturas en formato texto plano y HTML:

- Formato profesional con encabezados
- Tabla de productos con precios
- Cálculo de subtotales y totales
- Información del cliente
- Timestamp de la transacción

4.2.7. GeneradorPDF.kt - Facturas PDF

Utiliza iText7 para generar facturas profesionales en PDF:

- Diseño empresarial con encabezados
- Tablas formateadas con bordes y estilos
- Información detallada del cliente y productos
- Cálculos de totales con formato monetario
- Pie de página con información de contacto

4.2.8. ServicioCorreo.kt - Envío de Correos

Gestiona el envío de facturas por correo electrónico:

- Configuración mediante archivo properties
- Soporte para SMTP con autenticación
- Adjuntos de archivos PDF
- Manejo de errores de conexión
- Logging de operaciones de envío

4.2.9. Logger.kt - Sistema de Logging

Implementa un sistema de logging centralizado:

- Niveles de log: INFO, DEBUG, ERROR
- Timestamps automáticos
- Persistencia en archivo de log
- Formato estructurado para análisis
- Thread-safe para operaciones concurrentes

4.2.10. Validador.kt - Validaciones

Centraliza todas las validaciones de entrada:

- Validación de formato de email
- Validación de cantidades positivas

- Validación de strings no vacíos
- Validación de IDs de productos
- Mensajes de error descriptivos

4.2.11. Excepciones.kt - Manejo de Errores

Define excepciones personalizadas para el dominio:

- StockInsuficienteException: Cuando no hay stock disponible
- ProductoNoEncontradoException: Producto no existe en inventario
- EmailInvalidoException: Formato de email incorrecto
- CarritoVacioException: Intento de checkout con carrito vacío
- ErrorPersistenciaException: Fallos en lectura/escritura de archivos

5. Funcionalidades Implementadas

5.1. Registro de Clientes

El sistema solicita al usuario:

1. Nombre completo (validado como no vacío)
2. Correo electrónico (validado con expresión regular)

Las validaciones garantizan que los datos sean correctos antes de continuar con el proceso de compra.

5.2. Catálogo de Productos

El catálogo muestra en formato tabla:

- ID único del producto
- Nombre descriptivo
- Precio unitario en formato monetario
- Stock disponible real (considerando productos en carrito)

El stock mostrado es dinámico y se actualiza en tiempo real conforme se agregan productos al carrito, evitando que el usuario intente agregar más unidades de las disponibles.

5.3. Gestión del Carrito

5.3.1. Agregar Productos

El proceso de agregar productos incluye:

1. Selección del producto por ID
2. Validación de existencia del producto
3. Solicitud de cantidad deseada
4. Validación de stock disponible
5. Actualización del carrito
6. Visualización del carrito actualizado

Si el producto ya existe en el carrito, se incrementa su cantidad. El sistema valida que la cantidad total no exceda el stock disponible.

5.3.2. Eliminar Productos

El usuario puede eliminar productos del carrito:

1. Visualización de productos en carrito
2. Selección del producto a eliminar
3. Especificación de cantidad a eliminar
4. Actualización del carrito
5. Si la cantidad llega a cero, el producto se elimina completamente

5.4. Proceso de Checkout

El checkout incluye los siguientes pasos:

1. Validación de carrito no vacío
2. Presentación de resumen completo de la compra
3. Confirmación del usuario (S/N)
4. Si se confirma:
 - Actualización del inventario
 - Generación de facturas (TXT, HTML, PDF)
 - Intento de envío por correo
 - Registro en historial
 - Limpieza del carrito

5. Si se cancela, regresa al menú principal

5.5. Generación de Facturas

El sistema genera automáticamente tres tipos de facturas:

5.5.1. Factura TXT

Formato de texto plano con:

- Encabezado con nombre de la empresa
- Número de factura con timestamp
- Información del cliente
- Tabla de productos con cantidades y precios
- Subtotales y total general
- Pie de página con agradecimiento

5.5.2. Factura HTML

Formato HTML con estilos CSS:

- Diseño responsive
- Tabla estilizada con bordes y colores
- Formato monetario consistente
- Fácil visualización en navegadores
- Preparada para impresión

5.5.3. Factura PDF

Documento PDF profesional con:

- Diseño empresarial
- Tablas con formato profesional
- Tipografía clara y legible
- Información completa y estructurada
- Lista para envío o impresión

Todas las facturas se guardan en la carpeta `facturas/` con nomenclatura `FAC-YYYYMMDD_HHMMSS`.

5.6. Envío de Correos

El sistema intenta enviar la factura PDF por correo electrónico:

- Lee configuración de `config/mail.properties`
- Establece conexión SMTP con autenticación
- Adjunta la factura PDF
- Envía correo al cliente
- Registra el resultado en el log

Si el envío falla (por falta de configuración o error de conexión), el sistema continúa normalmente y las facturas quedan disponibles localmente.

5.7. Persistencia de Datos

5.7.1. Inventario (data/inventario.json)

El inventario se persiste en formato JSON con la siguiente estructura:

```
1  [  
2    {  
3      "id": 1,  
4      "nombre": "Laptop HP",  
5      "precio": 899.99,  
6      "stock": 15  
7    },  
8    ...  
9  ]
```

El archivo se actualiza automáticamente después de cada compra.

5.7.2. Historial (data/historial_compras.json)

El historial registra cada transacción:

```
1  [  
2    {  
3      "timestamp": "2026-03-29T00:47:29",  
4      "cliente": {  
5        "nombre": "Juan Perez",  
6        "email": "juan@example.com"  
7      },  
8      "productos": [...],  
9      "total": 1299.98  
10   }  
11  ]
```

5.8. Sistema de Logging

El sistema registra todas las operaciones importantes:

- Inicio y finalización de la aplicación
- Carga y guardado de inventario
- Agregado y eliminación de productos del carrito
- Confirmación y cancelación de compras
- Generación de facturas
- Envío de correos (éxito o fallo)
- Errores y excepciones

Los logs se guardan en `logs/app.log` con formato:

```
1 [2026-03-29 00:47:29] [INFO] Sistema iniciado
2 [2026-03-29 00:47:30] [INFO] Inventario cargado: 10 productos
3 [2026-03-29 00:48:15] [INFO] Producto agregado al carrito: Laptop HP
4 [2026-03-29 00:49:20] [ERROR] Error al enviar correo: Connection
  timeout
```

6. Principios de Diseño Aplicados

6.1. Principios SOLID

6.1.1. Single Responsibility Principle (SRP)

Cada clase tiene una única responsabilidad:

- `GestorInventario`: Solo gestiona persistencia de inventario
- `GeneradorFactura`: Solo genera facturas TXT/HTML
- `GeneradorPDF`: Solo genera facturas PDF
- `ServicioCorreo`: Solo envía correos
- `Logger`: Solo registra eventos
- `Validador`: Solo valida entradas

6.1.2. Open/Closed Principle (OCP)

El sistema está abierto a extensión pero cerrado a modificación:

- Nuevos formatos de factura pueden agregarse sin modificar existentes
- Nuevos tipos de productos pueden heredar de `Producto`
- Nuevas validaciones pueden agregarse sin modificar `Validador`

6.1.3. Liskov Substitution Principle (LSP)

Las clases derivadas pueden sustituir a sus clases base:

- `ProductoInventario` puede usarse donde se espera `Producto`

- `ProductoCarrito` mantiene el contrato de `Producto`

6.1.4. Interface Segregation Principle (ISP)

Las interfaces son específicas y no obligan a implementar métodos innecesarios. Cada componente expone solo los métodos que necesita.

6.1.5. Dependency Inversion Principle (DIP)

Las dependencias se inyectan y son configurables:

- `ServicioCorreo` depende de configuración externa
- `GestorInventario` puede cambiar el formato de persistencia
- Los componentes dependen de abstracciones, no de implementaciones concretas

6.2. Patrones de Diseño

6.2.1. Repository Pattern

Implementado en `GestorInventario` y `GestorHistorial`:

- Abstrae la lógica de persistencia
- Separa el acceso a datos de la lógica de negocio
- Facilita cambios en el mecanismo de almacenamiento

6.2.2. Factory Pattern

Utilizado en la creación del inventario inicial:

- Centraliza la lógica de creación de productos

- Facilita la inicialización del sistema
- Permite cambiar fácilmente el catálogo inicial

6.2.3. Builder Pattern

Aplicado en la generación de facturas:

- Construcción paso a paso de documentos complejos
- Separación entre construcción y representación
- Facilita la creación de diferentes formatos

6.2.4. Singleton Pattern

Implementado en `Logger`:

- Una única instancia del logger en toda la aplicación
- Acceso global al sistema de logging
- Garantiza consistencia en los registros

7. Manejo de Excepciones

El sistema implementa un manejo robusto de excepciones:

7.1. Excepciones Personalizadas

Se definieron excepciones específicas del dominio:

- `StockInsuficienteException`: Lanzada cuando se intenta agregar más productos de los disponibles

- `ProductoNoEncontradoException`: Cuando el ID del producto no existe
- `EmailInvalidoException`: Formato de email incorrecto
- `CarritoVacioException`: Intento de checkout sin productos
- `ErrorPersistenciaException`: Fallos en operaciones de archivo

7.2. Estrategia de Manejo

El manejo de excepciones sigue estas prácticas:

- **Captura específica**: Se capturan excepciones específicas, no genéricas
- **Mensajes descriptivos**: Cada excepción incluye un mensaje claro
- **Logging**: Todas las excepciones se registran en el log
- **Recuperación**: El sistema intenta recuperarse cuando es posible
- **Propagación**: Las excepciones se propagan cuando no pueden manejarse localmente

7.3. Validaciones Preventivas

Además de excepciones, se implementan validaciones preventivas:

- Validación de entrada antes de procesamiento
- Verificación de precondiciones en métodos críticos
- Comprobación de estado antes de operaciones
- Mensajes de error amigables para el usuario

8. Requisitos y Configuración

8.1. Requisitos del Sistema

8.1.1. Hardware

- Procesador: 1 GHz o superior
- RAM: Mínimo 512 MB
- Espacio en disco: 100 MB

8.1.2. Software

- Java Development Kit (JDK) 21 o superior
- Gradle 8.10.1 o superior (incluido via Gradle Wrapper)
- Sistema operativo: Windows, Linux o macOS

8.2. Instalación

8.2.1. Clonar el Repositorio

```
1 git clone https://github.com/marroquin9953/Proyecto-1---DSM941.git
2 cd proyecto_1
```

8.2.2. Verificar Java

```
1 java -version
```

Debe mostrar Java 21 o superior.

8.2.3. Dar Permisos (Linux/macOS)

```
1 chmod +x gradlew
```

8.3. Compilación

8.3.1. Compilar el Proyecto

```
1 ./gradlew build
```

En Windows:

```
1 gradlew.bat build
```

8.3.2. Generar JAR Ejecutable

```
1 ./gradlew shadowJar
```

El JAR se genera en: `app/build/libs/app-all.jar`

8.4. Ejecución

8.4.1. Opción 1: Con Gradle

```
1 ./gradlew run
```

8.4.2. Opción 2: JAR Directo

```
1 java -jar app/build/libs/app-all.jar
```

8.4.3. Opción 3: Desde IDE

Abrir el proyecto en IntelliJ IDEA o Android Studio y ejecutar la clase `AppKt`.

8.5. Configuración de Correo (Opcional)

Para habilitar el envío de facturas por correo, editar `config/mail.properties`:

```
1 mail.smtp.host=smtp.gmail.com
2 mail.smtp.port=587
3 mail.smtp.auth=true
4 mail.smtp.starttls.enable=true
5 mail.username=tu-correo@gmail.com
6 mail.password=tu-contrasena-de-aplicacion
7 mail.from=tu-correo@gmail.com
```

Para Gmail, generar una contraseña de aplicación en:

<https://myaccount.google.com/apppasswords>

9. Pruebas

9.1. Pruebas Unitarias

El proyecto incluye pruebas unitarias con JUnit 5:

9.1.1. Ejecutar Pruebas

```
1 ./gradlew test
```

9.1.2. Ver Reporte

El reporte HTML se genera en:

`app/build/reports/tests/test/index.html`

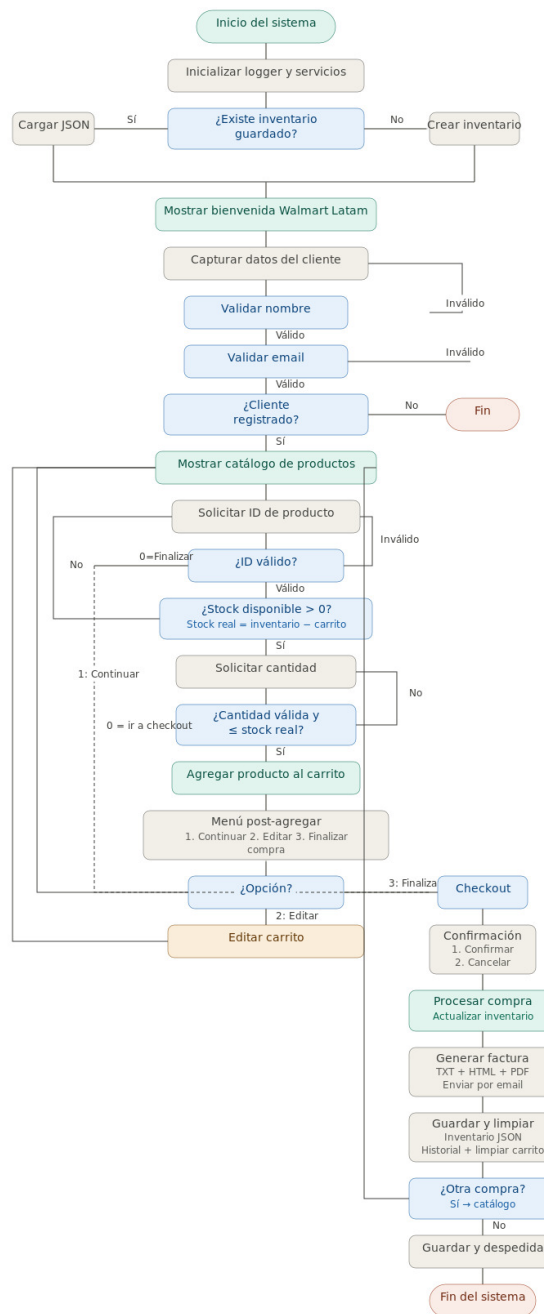
9.2. Áreas de Prueba

Las pruebas cubren:

- Validaciones de entrada
- Lógica del carrito de compras
- Cálculos de totales y subtotales
- Persistencia de datos
- Generación de facturas
- Manejo de excepciones

10. Flujo de Uso del Sistema

10.1. Diagrama de Flujo



El diagrama anterior ilustra el flujo completo del sistema desde el inicio hasta la finalización de una compra.

10.2. Ejemplo de Uso

10.2.1. Escenario: Compra de Laptop y Mouse

1. Usuario inicia el sistema
2. Ingresa nombre: "Juan Pérez"
3. Ingresa email: "juan.perez@example.com"
4. Ve el catálogo con 10 productos
5. Selecciona producto ID 1 (Laptop HP) - Cantidad: 1
6. Selecciona producto ID 3 (Mouse Logitech) - Cantidad: 2
7. Revisa el carrito: Total \$949.97
8. Confirma la compra
9. Sistema genera facturas y envía correo
10. Compra completada exitosamente

11. Limitaciones y Mejoras Futuras

11.1. Limitaciones Conocidas

- **Interfaz de consola:** No tiene interfaz gráfica, limitando la experiencia de usuario

- **Configuración manual:** El envío de correo requiere configuración manual del archivo `properties`
- **Sin autenticación:** No hay sistema de usuarios con contraseñas
- **Persistencia local:** El inventario se almacena en archivos JSON, no en base de datos
- **Moneda única:** Solo soporta dólares estadounidenses
- **Sin descuentos:** No implementa sistema de promociones o cupones
- **Sin pagos reales:** No integra pasarelas de pago

11.2. Mejoras Futuras

11.2.1. Corto Plazo

- Implementar sistema de descuentos y promociones
- Agregar soporte para múltiples monedas
- Mejorar la interfaz de consola con colores y formato
- Implementar búsqueda de productos por nombre
- Agregar categorías de productos

11.2.2. Mediano Plazo

- Desarrollar interfaz gráfica con JavaFX o Compose Desktop
- Migrar a base de datos relacional (PostgreSQL, MySQL)
- Implementar autenticación y autorización de usuarios

- Sistema de roles (cliente, administrador, vendedor)
- Reportes y estadísticas de ventas
- Dashboard administrativo

11.2.3. Largo Plazo

- Integración con pasarelas de pago (PayPal, Stripe)
- API REST para integración con otros sistemas
- Aplicación móvil nativa (Android/iOS)
- Sistema de recomendaciones basado en IA
- Notificaciones push en tiempo real
- Soporte multidioma
- Sistema de reviews y calificaciones
- Integración con sistemas de envío
- Chat en vivo para soporte al cliente